

REPORTE DE PRÁCTICA DE MULTIMEDIA

Práctica 6: Esteganografía Segura con Cifrado XOR y Semilla Aleatoria

Estudiante: Fragoso Rizo Jose Manuel

Materia: Multimedia

Fecha: 21 de febrero de 2026

Proyecto: Implementación de Esteganografía determinista mediante Claves Maestras

1. Introducción

A diferencia de la esteganografía básica, esta práctica implementa una capa de seguridad avanzada. El programa no elige posiciones al azar de forma caótica, sino de forma **determinista** mediante una clave. Esto garantiza que solo quien posea la clave exacta pueda reconstruir la secuencia de píxeles y descifrar el mensaje.

2. Metodología Técnica

El sistema utiliza tres pilares de seguridad multimedia:

1. **Hashing del Password:** Se aplica la función **SHA-256** a la clave elegida para generar una huella digital única.
2. **Generador Pseudoaleatorio:** La huella se usa como **semilla (seed)** para que el generador de números aleatorios seleccione siempre los mismos píxeles en el mismo orden.
3. **Cifrado XOR:** Antes de ocultar el mensaje, los bytes se someten a una operación XOR con el hash de la clave, haciendo que los datos sean ilegibles incluso si se extraen los bits.

3. Código Fuente (Secure Steganography)

```
# =====
# Autor: Fragoso Rizo Jose Manuel
# Materia: Multimedia - Práctica 6
# =====
import struct, os, math, hashlib, random

def leer_bmp(filepath):
    with open(filepath, 'rb') as f:
        data = f.read()
        offset = struct.unpack_from('<I', data, 10)[0] #
        width = struct.unpack_from('<i', data, 18)[0]
        height = struct.unpack_from('<i', data, 22)[0]
        header = bytearray(data[:offset])
```

```

        pixels = bytearray(data[offset:])
        return header, pixels, width, height

def guardar_bmp(filepath, header, pixels):
    with open(filepath, 'wb') as f:
        f.write(header)
        f.write(pixels) #

def cifrar_descifrar_xor(datos, password):
    hash_base = hashlib.sha256(password.encode()).digest() #
    clave_extendida = (hash_base * (len(datos) // len(hash_base) +
1))[ :len(datos)]
    return bytes(a ^ b for a, b in zip(datos, clave_extendida))

def obtener_posiciones_aleatorias(total_pixeles, cantidad_bits, password):
    semilla = int(hashlib.sha256(password.encode()).hexdigest(), 16) % (10**8) #
    random.seed(semilla)
    return random.sample(range(total_pixeles), cantidad_bits)

def embed_secure(src_path, dst_path, mensaje, password):
    header, pixels, w, h = leer_bmp(src_path)
    msg_bytes = mensaje.encode('utf-8')
    msg_cifrado = cifrar_descifrar_xor(msg_bytes, password) #

    # Preparar datos: 32 bits para longitud + mensaje cifrado
    datos_a_esconder = struct.pack('>I', len(msg_bytes)) + msg_cifrado
    bits = []
    for byte in datos_a_esconder:
        for i in range(7, -1, -1):
            bits.append((byte >> i) & 1)

    posiciones = obtener_posiciones_aleatorias(len(pixels), len(bits), password)
    pixels_mod = bytearray(pixels)
    for i, pos in enumerate(posiciones):
        pixels_mod[pos] = (pixels_mod[pos] & 0xFE) | bits[i] #

    guardar_bmp(dst_path, header, pixels_mod)

def extract_secure(stego_path, password):
    header, pixels, w, h = leer_bmp(stego_path)
    # Extraer primero los 32 bits de longitud
    pos_longitud = obtener_posiciones_aleatorias(len(pixels), 32, password) #
    len_bits = [pixels[pos] & 1 for pos in pos_longitud]

    msg_len = 0

```

```

for b in len_bits: msg_len = (msg_len << 1) | b #

total_bits = 32 + (msg_len * 8)
todas_las_pos = obtener_posiciones_aleatorias(len(pixels), total_bits,
password)
pos_mensaje = todas_las_pos[32:]

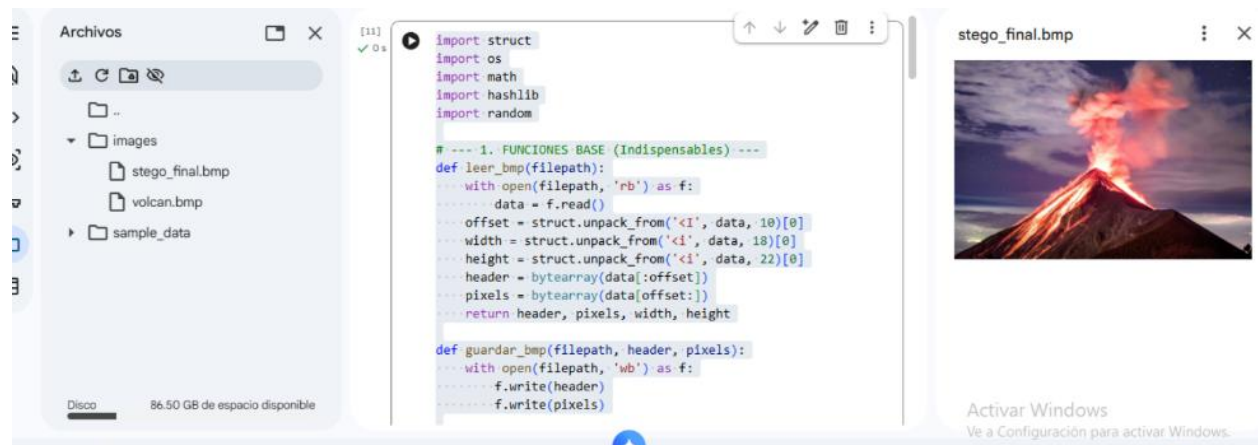
bits_mensaje = [pixels[pos] & 1 for pos in pos_mensaje]
msg_cifrado = bytearray()
for i in range(0, len(bits_mensaje), 8):
    byte = 0
    for bit in bits_mensaje[i:i+8]: byte = (byte << 1) | bit
    msg_cifrado.append(byte)

return cifrar_descifrar_xor(msg_cifrado, password).decode('utf-8') #

```

4. Resultados (Ejecución Final)

- **Imagen Original:** volcan.bmp.
- **Archivo Stego:** stego_final.bmp.
- **Mensaje:** "Mensaje Cifrado UPIITA 2026".
- **Clave:** "mi_llave_maestra".
- **Calidad Visual:** Al ser un método LSB determinista, la imagen procesada es visualmente idéntica a la original, manteniendo una alta fidelidad (PSNR).



5. Conclusión

Esta práctica demuestra que la seguridad en multimedia no solo depende de ocultar el dato, sino de la robustez del algoritmo. Gracias al uso de SHA-256 y semillas aleatorias,

el mensaje permanece protegido contra ataques de extracción masiva, estableciendo un estándar de comunicación segura sobre canales visuales.
